



First semester 2022/2023
Advanced topics in software development
Homework Assignment # 1
Design patterns
ETA: 28 Nov midnight.

Create by : Ahmad istatieh

Under supervision

Dr. Mustafa Assaf
En.Mohammad Qzeh

Before the Refactor How the System Works Now

We have a system called TaskMaster Processing System (TMPS).

This system is designed to execute different types of tasks such as generating reports, sending emails, or processing data based on what the user requests.

Let's imagine a user named Ahmad who wants to generate a monthly report.

As soon as Ahmad requests the report, the system immediately goes to the TemplateManager to build the job template. The problem is that the TemplateManager starts from scratch every single time. It performs a heavy process that simulates loading data, reading files, and preparing large content and this whole operation takes around 3 seconds each time.

After the TemplateManager finishes building the heavy template, it returns a new template to the system.

This template then creates a completely new Job object that contains the job type (REPORT), its name, configuration settings, and a new unique ID.

The system then assigns this Job to Ahmad and prepares it for execution.

Here the JobExecutor takes over: it receives the Job and tries to execute it using a long series of if/else statements inside the class to determine what type of job it is and how to run it.

Before executing, the executor requests a new connection from the ConnectionManager, which also creates a new connection every time without any reuse.

The problem is that if Ahmad requests another report, or any other task like an email or a data processing job, the system repeats the entire process from the beginning.

Key Problems in This Code :

Issue 1. Heavy Template Rebuilding (Performance Problem) :

One of the first problems that appears clearly in the current implementation is the way templates are created inside the TemplateManager class. Every time the system wants to generate a job, the TemplateManager builds a heavy template from scratch, even if it's the same exact report or email that was requested previously. The template-building process simulates expensive operations such as loading large data, parsing files, and generating content, which causes a noticeable delay of about three seconds during every request.

can be seen in the following code:

```
private String simulateHeavyLoad(String msg) {
    System.out.println("Simulating heavy template creation for: " + msg);
    try { Thread.sleep(3000); } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
    return "Large template";
}
```

Because this heavy work happens on every single job creation, the system becomes extremely slow, especially when users request similar tasks multiple times. The current design does not benefit from any form of reuse, causing noticeable delays and wasted resources.

Issue 2.No Real Connection Pool (Inefficient Resource Management) :

A major issue exists in the current ConnectionManager. The system creates a new database connection for every job and does not reuse existing ones. Even worse, once the limit of 10 connections is exceeded, the method prints a warning and returns null, which can easily lead to system failures.

can be seen in the following code:

```
public Connection createConnection() {
    int n = ++counter;
    if (n > MAX) {
        //TODO problem 2: we need to ensure no more 10 connections.
        System.out.println("Warning: too many connections created. Still creating (naive).");
        return null;
    }
    return new Connection("Conn-" + n);
}
```

In a real system, creating and destroying database connections repeatedly is extremely expensive. Because this manager does not implement a Connection Pool, there is no reuse, no blocking mechanism, and no queue for waiting tasks. This leads to wasted resources and potential crashes when connections run out.

Issue 3.JobExecutor Violates SRP (Too Many Responsibilities) :

One of the key design problems appears in the JobExecutor class, especially in the executeJob method. Instead of having a single, clear responsibility, this method is doing many different things at the same time. It prints logs, creates a database connection, decides which job type to run, calls the specific logic for each job type, and then closes the connection and prints a “finished” message. All of these concerns are mixed together inside one place.

can be seen in the following code:

```
public void executeJob(Job job) {
    System.out.printf("[NaiveExecutor] Starting job %s (%s) requested by %s%n",
        job.getName(), job.getType(),
        job.getRequestedBy() == null ? "unknown" : job.getRequestedBy().getName());
    // Acquire connection directly (no pool reuse)
    Connection c = cm.createConnection();
    try {
        if ("EMAIL".equals(job.getType())) {
            executeEmailJob(job, c);
        } else if ("DATA".equals(job.getType())) {
            executeDataJob(job, c);
        } else if ("REPORT".equals(job.getType())) {
            executeReportJob(job, c);
        } else {
            System.out.println("Unknown job type - nothing done.");
        }
    } finally {
        cm.closeConnection(c);
        System.out.printf("[NaiveExecutor] Finished job %s%n", job.getName());
    }
}
```

In this single method, the executor is responsible for logging, connection management, job-type selection, and delegating to the detailed logic of each job. This clearly violates the Single Responsibility Principle, because any change in logging, connection handling, or job execution logic will require modifying the same method. Over time, this method can easily keep growing and becomes very hard to maintain.

Issue 4. Long if/else Chain for Job Type (OCP Violation)

A very clear design flaw appears in the `executeJob` method, where the system decides how to run a job through a long chain of nested if/else statements. Each condition checks the job type as a string and calls a specific private method.

can be seen in the following code:

```

if ("EMAIL".equals(job.getType())) {
    executeEmailJob(job, c);
} else if ("DATA".equals(job.getType())) {
    executeDataJob(job, c);
} else if ("REPORT".equals(job.getType())) {
    executeReportJob(job, c);
} else {
    System.out.println("Unknown job type – nothing done.");
}

```

This method violates the Open-Closed Principle, because adding any new job type requires modifying this method directly. The logic is not extendable, and the executor becomes tied to every job type in the system.

Issue 5.Lack of Permission Validation (Security Weakness)

Another issue appears with job authorization. While the User class does contain a permission list, the current system never checks whether the user is actually allowed to execute the requested job type.

For example, regardless of the user's permissions, this line runs the job without verification:

```

executor.executeJob(reportJob);

```

This makes the system vulnerable, because any user can trigger any job type, even ones they should not have access to.

Issue 6.Connection Closing Does Nothing :

The closeConnection method inside ConnectionManager is misleading and non-functional. Although the method name suggests that the connection is being returned or cleaned up, all it does is print a message.

can be seen in the following code:

```
public void closeConnection(Connection c) {  
    System.out.printf("[ConnectionManager] Closing connection %s (naive).%n", c.getId());  
    // naive: does nothing to reuse  
}
```

This means connections are never actually closed or returned, making the connection counter continuously increase until it exceeds the limit.

Issue 7.Strong Coupling Between Components :

Many classes directly instantiate or reference each other without interfaces or abstractions. For example, JobExecutor depends directly on ConnectionManager, making it impossible to replace or swap the connection mechanism without rewriting the executor.

```
private final ConnectionManager cm;
```

This strong coupling reduces modularity and makes the system rigid.

After the Refactor How the System Works Now

The system no longer rebuilds everything from scratch; instead, it uses design patterns to make execution faster, safer, and more organized.

Let's go back to our example with Ahmad. Ahmad wants to generate the same monthly report.

But this time, instead of going to the TemplateManager and spending three seconds building a heavy template every single time, the system goes to the JobTemplateRegistry, where a ready-made prototype of the report template is already stored.

The system simply clones this prototype to create a new Job instantly no heavy loading, no delays.

After the Job is created, it is passed to the JobExecutorProxy instead of the raw executor. The Proxy first checks whether Ahmad has permission to execute this type of job. If he is authorized, the Proxy logs the start time, measures execution time, and retrieves a connection from the Connection Pool, which reuses existing database connections instead of creating new ones every time.

Once everything is ready, the Proxy forwards the Job to the real executor.

The JobExecutor no longer uses if/else chains. Instead, it selects the correct behavior through the Strategy Pattern, meaning each job type (REPORT, EMAIL, DATA) has its own execution strategy.

After the job completes, the Proxy logs the end of execution, releases the connection back into the pool, and reports how long the operation took.

Now, if Ahmad requests another report, or any other task, the system performs the process efficiently: No rebuilding templates , No waiting three seconds , No creating unnecessary database connections , And no insecure or unorganized execution flow

How Each Problem Was Solved :

Issue 1: Heavy Template Rebuilding (Solved with Prototype + Registry Pattern) :

In the refactored version, the TemplateManager no longer rebuilds heavy templates from scratch.

Instead, we introduced the Prototype Pattern along with a Template Registry.

The heavy template is created only once, registered in the JobTemplateRegistry, and then reused by cloning.

Now, creating a new job takes milliseconds instead of three seconds.

Example (fixed code):



```
1 JobTemplateRegistry registry = new JobTemplateRegistry();
2
3 registry.registerTemplate("MONTHLY_REPORT",
4     new ReportJobTemplate("MonthlyReport", "format=PDF;brand=TaskMaster", "Large
5     template body"));
6
7 Job reportJob = registry.createJobFromTemplate("MONTHLY_REPORT");
```

This eliminates the expensive heavy-load simulation and ensures extremely fast job creation.

Issue 2: No Real Connection Pool (Solved with Object Pool Pattern) :

Previously, every job created a new DB connection, and once the limit reached 10, the system failed.

Now we use a Connection Pool, which reuses existing connections and releases them back after execution.

Example (fixed code):

```
Untitled-1

1  ConnectionPool pool = new SimpleConnectionPool(10);
2
3  Connection conn = pool.acquire();
4  // execute job
5  pool.release(conn);
```

This prevents connection exhaustion, reduces overhead, and makes the system scalable.

Issue 3: JobExecutor Violates SRP (Solved with Strategy Pattern + Proxy) :

Originally, JobExecutor did everything in one place:

Logging , Choosing job type , Opening connections , Executing logic , Closing connections

Now responsibilities are cleanly separated:

JobExecutor - only executes using a strategy

JobExecutorProxy - handles logging, timing, permissions, connection usage

JobStrategy - each job has its own execution logic

Example (fixed code):

```
Untitled-1

1  JobStrategy strategy = strategyFactory.getStrategy(job.getType());
2  strategy.execute(job, connection);
```

This makes the executor clean, maintainable, and easy to extend.

Issue 4: Long if/else Chain for Job Types (Solved with Strategy + StrategyFactory) :

The refactored system removes all if/else chains.

Each job type now has its own JobStrategy implementation.

Example (fixed code):


```
Untitled-1

1 strategyFactory.register("EMAIL", new EmailJobStrategy());
2 strategyFactory.register("REPORT", new ReportGenerationStrategy());
3 strategyFactory.register("DATA", new DataProcessingStrategy());
4
5 JobStrategy strategy = strategyFactory.getStrategy(job.getType());
```

Adding a new job type now requires adding one new strategy class, without touching existing code.

Issue 5: No Permission Validation (Solved with Proxy Authorization Check) :

Previously, any user could execute any job - even if they had no permissions.
Now the JobExecutorProxy checks permissions before execution.

Example (fixed code):

```
Untitled-1

1 if (!job.getRequestedBy().hasPermission(job.getType())) {
2     System.out.println("Permission Denied for user: " +
3     job.getRequestedBy().getUsername());
4     return;
5 }
6
```

This ensures secure execution and proper access control.

Issue 6: Connection Closing Does Nothing (Solved with Real Release into Pool) :

Before the refactor, closeConnection() printed a message but didn't actually close or reuse anything.

Now the proxy returns the connection to the pool after execution.

Example (fixed code):

A code editor window with a dark background and light gray text. The title bar at the top says "Untitled-1". There are three small circular window control buttons on the left. The code is on a single line, numbered "1" in the left margin: `pool.release(connection);`

Connections are no longer leaked or wasted.

Issue 7: Strong Coupling Between Components (Solved with Interfaces + Abstractions + Patterns) :

Originally, classes depended directly on concrete implementations like `ConnectionManager`. After the refactor, we introduced interfaces such as: `ConnectionPool` , `JobPrototype` , `JobStrategy`

And used Factory + Proxy + Prototype + Strategy to reduce coupling.

Example (fixed code):

A code editor window with a dark background and light gray text. The title bar at the top says "Untitled-1". There are three small circular window control buttons on the left. The code is on two lines, numbered "1" and "2" in the left margin: `1 private final ConnectionPool pool;` and `2 private final JobStrategyFactory strategyFactory;`

This makes the system flexible, modular, and easy to extend without breaking existing code.

Benefits of the Refactored Design :

1. Major Performance Improvement (Thanks to Prototype Pattern)

The original system rebuilt heavy templates every time a job was created, causing a 3second delay per request.

In the refactored design, heavy templates are created once, registered in the `JobTemplateRegistry`, and then instantly cloned.

Benefit: Job creation is now nearly instantaneous. There is no repeated loading, no waiting time, and the overall performance of the system is drastically improved.

2. Better Resource Efficiency Through Connection Pooling

Previously, each job opened a brand-new database connection and never truly closed it, which could lead to resource exhaustion.

The new design uses an Object Pool, allowing connections to be reused safely.

Benefit: No connection leaks , No system crashes because of connection limits , Better performance under load , Efficient use of system resources

3. Clean Separation of Responsibilities (SRP Applied)

Before the refactor, the JobExecutor class handled logging, permissions, connection creation, execution logic, and cleanup all in one method.

Benefit: The system becomes easier to maintain, modify, debug, and extend without breaking other parts of the code.

4. High Extensibility (No More if/else Chains)

Adding a new job type in the old system required modifying the executor class directly, breaking the OpenClosed Principle.

With the Strategy Pattern + StrategyFactory, each job type has its own class.

Benefit: Adding new job types is now effortless:
just create a new strategy class and register it no modifications to existing code.

5. Improved Security Through Permission Validation

Before the refactor, any user could execute any job type without authorization.

Now, the Proxy checks user permissions before execution, blocking unauthorized job types.

Benefit: The system becomes more secure and follows proper access-control rules.

6. Reusable and Maintainable Architecture

Each component is now independent and loosely coupled.

Benefit: The system is more maintainable, testable, and adaptable to future changes.

Design Decisions :

1. Choosing the Prototype Pattern for Job Template Creation :

The old system rebuilt heavy job templates from scratch every time a job was created. This caused a 3second delay per request and wasted resources.

Job templates do not change frequently, so recreating them repeatedly was unnecessary.

The Prototype Pattern allows us to: Create heavy templates once , Store them inside a registry , Produce new job instances by cloning in milliseconds

This eliminates delays, reduces CPU usage, and makes job creation extremely fast.

2. Using a Template Registry for Managing Prototypes :

We needed a centralized and organized place to store and retrieve job templates.

Benefits : Easy to register all templates in one place , No need to search code or rebuild templates , Clean and scalable architecture

3. The Object Pool Pattern for Database Connections :

The original ConnectionManager created a new connection for every request and never truly closed them. This could cause:

Connection exhaustion , System crashes , High overhead

Object Pool solves the problem as :

Creates a limited number of reusable connections , Reuses them safely , Prevents leaks and improves performance

4. Applying the Strategy Pattern for Job Execution Logic :

The old executor used a long if/else chain

This violated the OpenClosed Principle, because adding a new job required editing the executor.

Strategy has Each job type has a different execution behavior, so we give each behavior its own class Report , Email , DataProcessing

This eliminates if/else and makes the system flexible.

5. The Proxy Pattern for Authorization, Logging, and Connection Handling

The old executor handled too many responsibilities

The Proxy sits in front of the executor and handles:

Authorization , Logging , Audit , Timing , Connection acquire , Connection release

Then the Proxy calls the real executor for the actual job logic.

6. Introducing Interfaces to Reduce Coupling

The original system tightly coupled classes to each other.

We introduced interfaces like: JobStrategy , JobPrototype , ConnectionPool

Benefits : Low coupling , High flexibility , Easy code replacement

The output of the TMS :

```
TMPs App

Run REPORT job (Monthly Report)
Connection Opened | ID = Pool-Conn-1

AuditLog -----
Job ID   : REPORT-1764603867797
Owner    : alice
Job Type : REPORT
-----

Executing Report Job: MonthlyReport
Connection Closed | ID = Pool-Conn-1

-----

Run EMAIL job (Monthly Email Report)
Connection Opened | ID = Pool-Conn-1

AuditLog -----
Job ID   : EMAIL-1764603867808
Owner    : alice
Job Type : EMAIL
-----

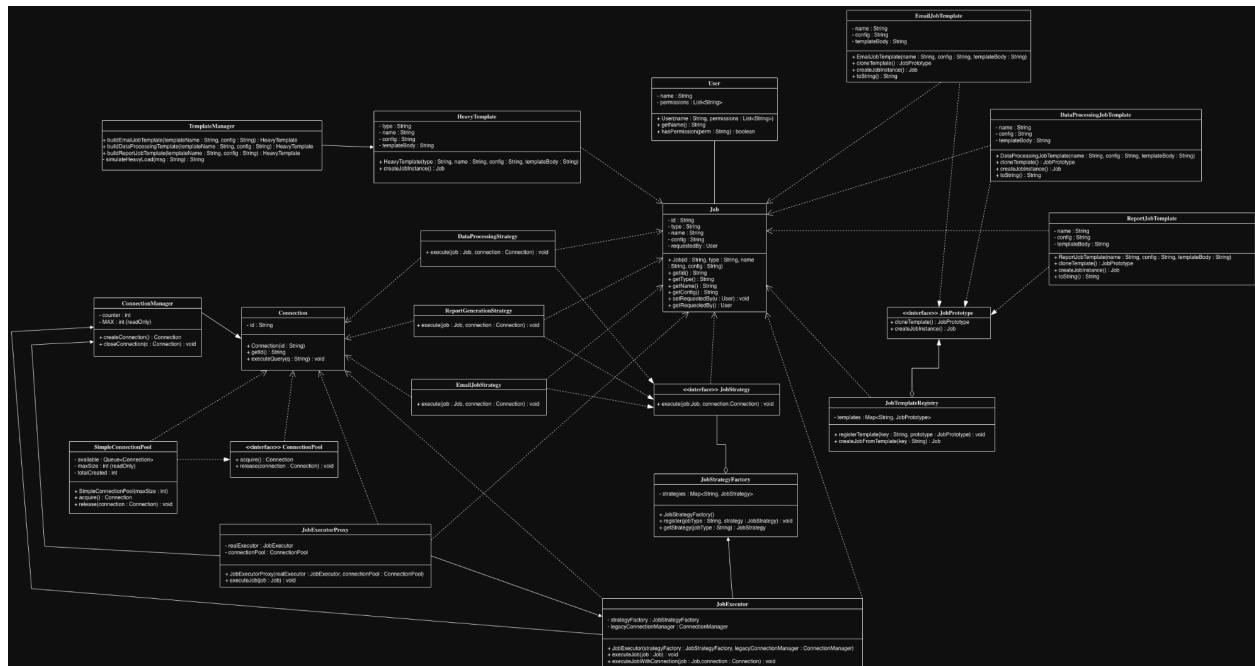
Email Job Executed: Monthly email report
Connection Closed | ID = Pool-Conn-1

-----

Run DATA job (should be denied)
Permission Denied for user: alice
AUTHORIZATION FAILED for job type: DATA

Process finished with exit code 0
```

The UML :



In the end : the refactoring of the TaskMaster Processing System (TMPS) significantly improved the structure , performance , and maintainability of the application. The old implementation suffered from heavy template rebuilding , inefficient connection handling , and tightly coupled components that were difficult to extend. By introducing the Prototype , Strategy , Proxy , and Object Pool design patterns , the system became faster , more organized , and easier to maintain. The new architecture provides a solid foundation for future extensions and ensures stable and secure job execution.